

OPTIMIZACIÓN DE COSTOS

Para este workflow se optó por utilizar modelos de **OpenAI (ChatGPT)** en lugar de Claude, principalmente por las características del proceso y la necesidad de obtener respuestas en tiempo real.

El flujo funciona como un chatbot de atención al cliente a través de WhatsApp. Cada mensaje recibido debe ser procesado *inmediatamente* para extraer información, registrar o actualizar el contacto en *Airtable* y generar una posible respuesta para el cliente. Debido a esta dinámica, se priorizó un modelo que *pueda utilizarse mediante API con baja latencia y procesar cada interacción de manera individual*.

En este caso no resulta conveniente utilizar una **API de procesamiento por lotes (Batch)** como estrategia principal de ahorro. El procesamiento Batch es especialmente útil para tareas masivas y no urgentes, donde pueden procesarse de forma diferida a un costo menor. Sin embargo, en un chatbot de WhatsApp introducir esa espera afectaría directamente el tiempo de respuesta y la experiencia del usuario. Por este motivo priorice la velocidad antes que ahorro de costos.

A su vez, el workflow utiliza dos agentes de IA con tareas distintas. El primero procesa la información del mensaje que envía el cliente y en caso de ser posible trata de vincularlo con uno de los productos de la lista de *Airtable*. Mientras que el segundo genera una propuesta de respuesta al cliente.

Para optimizar los costos de ejecución se estableció un **límite máximo de tokens de salida diferenciados** para cada agente.

Teniendo en cuenta que el primero realiza una tarea sencilla de clasificación/extracción le asigne un máximo de únicamente 50 tokens; un modelo económico y un prompt acotado. Mientras que al agente encargado de generar la respuesta al cliente le puse un límite de 1000 tokens ya que debe consultar información almacenada en *Airtable* y redactar una respuesta natural y contextualizada. Es un límite bajo pero más elevado que el del primer agente.

Esta diferenciación permite reducir el consumo innecesario de tokens y, por lo tanto, el costo por ejecución.

También se incorporó una **memoria simple asociada a la conversación** únicamente del segundo agente. Al que solo extrae el producto no le aporta prácticamente nada y puede aumentar los tokens de contexto.

Esta memoria simple permite conservar contexto de interacciones anteriores del mismo cliente, evitando que tenga que reconstruir manualmente todo el contexto en cada ejecución y permite generar respuestas más coherentes a lo largo de una conversación. Al mismo tiempo, la memoria debe mantenerse acotada para evitar que el crecimiento del contexto incremente innecesariamente la cantidad de tokens de entrada y, por lo tanto, el costo de cada llamada al modelo.

En cuanto al modelo, en el primer agente elegí **openai/gpt-oss-20b**.

Elegí este porque se trata de una tarea simple y estructurada que no requiere un modelo

de alta capacidad. Se priorizó un modelo de menor costo y baja latencia. No se utilizó GPT-OSS 120B porque su mayor capacidad no aporta un beneficio significativo para esta tarea y su costo por token es aproximadamente el doble.

En cambio para el segundo modelo elegí **openai/gpt-oss-120b** ya que es una tarea más compleja y es el que más impacto va a tener en la calidad del workflow.

MATRIZ DE DECISIÓN- Cuadro comparativo

Tarea	Estrategia elegida	Justificación	Optimización de costos
Clasificación y extracción de información	Modelo de OpenAI vía API	Tarea breve que requiere procesamiento inmediato	Max Tokens limitado; modelo económico y prompt acotado
Generación de respuesta al cliente	Modelo de OpenAI vía API	Necesidad de baja latencia y velocidad de respuesta	Límite de tokens de salida + memoria simple
Contexto de conversaciones anteriores	Simple Memory	Permite mantener continuidad entre mensajes del mismo cliente. Cumpliría una función parecida a la de prompt caching de Claude	Memoria acotada para controlar tokens de entrada
Procesamiento Batch	No utilizado	El chatbot requiere respuesta en tiempo real; el procesamiento diferido no es adecuado para esta tarea. Sería conveniente para grandes volúmenes de documentación que no requieren respuesta inmediata	Se prioriza velocidad sobre el descuento disponible por procesamiento Batch

Por lo tanto, la elección no se basa únicamente en el costo nominal por token, sino en la relación entre **costo, velocidad, volumen y criticidad de la tarea**. Para este caso de uso, el procesamiento individual en tiempo real ofrece un mejor equilibrio que una arquitectura basada en procesamiento Batch.

El ahorro se obtiene principalmente mediante el **control del número de tokens**, prompts específicos, respuestas breves y una memoria limitada. Además, podría utilizarse un modelo de menor costo para la primera tarea —clasificación y extracción de información— y reservar un modelo de mayor capacidad únicamente para la generación de respuestas cuando la complejidad de la consulta lo requiera.

AHORRO ESTIMADO

Actualmente, gpt-oss-20b cuesta USD 0,075/M tokens de entrada y USD 0,30/M de salida, mientras que gpt-oss-120b cuesta USD 0,15/M de entrada y USD 0,60/M de salida.

Sin optimización, si ambos agentes utilizaran GPT-OSS-120B con un máximo de 1.000 tokens cada uno:

$$2 \times 1.000 \times \text{USD } 0,60 / 1.000.000 = \text{USD } 0,0012 \text{ por conversación}$$

Por cada 1.000 conversaciones:

USD 1,20

Con esta arquitectura:

USD 0,615

Por lo tanto:

$$\text{Ahorro} = \text{USD } 1,20 - \text{USD } 0,615 = \text{USD } 0,585$$

Eso equivale a un ahorro máximo teórico del 48,75% en tokens de salida.

Este cálculo representa un escenario máximo, ya que el parámetro Max Tokens establece un límite y no implica que el modelo consuma siempre esa cantidad. En la práctica, las respuestas pueden finalizar utilizando menos tokens, por lo que el costo real dependerá del consumo efectivo de cada ejecución.